

Complete Project Setup Guide - CCTV Motion Detection System

Project Structure Overview

cctv-motion-detection/

Root folder

└─ backend-nextjs/

Next.js Backend & Web Dashboard

└─ python-motion-service/

Python Motion Detection Service

└─ mobile-app/

React Native Mobile App

1. NEXT.JS BACKEND & WEB DASHBOARD

Create Next.js Project

bash

npx create-next-app@latest backend-nextjs

cd backend-nextjs

Choose: TypeScript (Yes), Tailwind CSS (Yes), App Router (Yes)

Complete Folder Structure

backend-nextjs/

└─ app/

│ └─ api/

│ │ └─ auth/

│ │ │ └─ login/

│ │ │ │ └─ route.ts

│ │ │ └─ register/

│ │ │ │ └─ route.ts

│ │ │ └─ logout/

│ │ │ │ └─ route.ts

│ │ └─ motion/

│ │ │ └─ route.ts

│ │ │ # Motion detection API (Code: nextjs_motion_api)

│ │ └─ cameras/

│ │ │ └─ route.ts

│ │ │ # GET all cameras, POST create camera

│ │ │ └─ [id]/

│ │ │ │ └─ route.ts

│ │ │ # GET, PUT, DELETE specific camera

│ └─ shops/

│ └─ route.ts



```

├── table.tsx
├── dashboard/
│   ├── Sidebar.tsx
│   ├── Header.tsx
│   ├── StatsCard.tsx
│   └── EventCard.tsx
├── camera/
│   ├── CameraList.tsx
│   ├── CameraCard.tsx
│   ├── LiveView.tsx
│   └── CameraSettings.tsx
├── notifications/
│   └── NotificationBell.tsx
├── lib/
│   ├── database.ts           # Database helper functions (Code: database_helpers)
│   ├── notifications.ts      # Push notification service (Code: notification_service)
│   ├── auth.ts               # Authentication utilities
│   └── utils.ts               # General utilities
├── prisma/
│   ├── schema.prisma         # Database schema (Code: database_schema)
│   └── migrations/
├── public/
│   ├── uploads/
│   │   └── motion/           # Motion detection images
│   ├── audio/
│   │   └── welcome_message.mp3 # Audio message file
│   └── images/
├── types/
│   ├── index.ts              # TypeScript type definitions
│   └── api.ts
├── middleware.ts              # Auth middleware
├── .env                       # Environment variables
├── .env.example
├── next.config.js
├── package.json
├── tsconfig.json
└── README.md

```

Install Dependencies

```
bash
```

```
cd backend-nextjs
```

Core dependencies

```
npm install @prisma/client firebase-admin bcryptjs jsonwebtoken
```

```
npm install next-auth
```

Development dependencies

```
npm install -D prisma typescript @types/node @types/react
```

```
npm install -D @types/bcryptjs @types/jsonwebtoken
```

UI dependencies (optional)

```
npm install lucide-react clsx tailwind-merge
```

```
npm install @radix-ui/react-dialog @radix-ui/react-dropdown-menu
```

Initialize Prisma

```
bash
```

```
npx prisma init
```

This creates prisma/schema.prisma and .env

Create Files

Step 1: Copy database schema

```
bash
```

Copy the code from artifact "database_schema" to:

prisma/schema.prisma

Step 2: Copy database helpers

```
bash
```

Copy the code from artifact "database_helpers" to:

lib/database.ts

Step 3: Copy notification service

```
bash
```

Copy the code from artifact "notification_service" to:

lib/notifications.ts

Step 4: Copy motion API route

```
bash

# Copy the code from artifact "nextjs_motion_api" to:
# app/api/motion/route.ts
```

2. PYTHON MOTION DETECTION SERVICE

Create Python Project

```
bash

mkdir python-motion-service
cd python-motion-service
```

Complete Folder Structure

```
python-motion-service/
├── src/
│   ├── __init__.py
│   ├── motion_detector.py      # Main detector class (Code: motion_detection_py)
│   ├── camera_manager.py      # Camera connection manager
│   ├── audio_player.py        # Audio playback handler
│   └── api_client.py          # API communication
├── config/
│   ├── __init__.py
│   ├── settings.py            # Configuration settings
│   └── camera_config.json     # Camera configurations
├── audio/
│   └── welcome_message.mp3    # Audio message
├── models/                    # Optional: AI models
│   └── person_detection.pth
├── logs/
│   └── motion_detector.log
├── tests/
│   ├── __init__.py
│   ├── test_motion_detector.py
│   └── test_api_client.py
├── scripts/
│   └── generate_audio.py      # Generate TTS audio
```

```
| └─ test_camera.py          # Test camera connection
| └─ requirements.txt
| └─ Dockerfile
| └─ docker-compose.yml
| └─ .env
| └─ .env.example
| └─ main.py                 # Entry point
└─ README.md
```

Create Virtual Environment & Install Dependencies

```
bash

# Create virtual environment
python -m venv venv

# Activate (Windows)
venv\Scripts\activate

# Activate (Mac/Linux)
source venv/bin/activate

# Install dependencies
pip install opencv-python numpy requests pygame python-dotenv
pip install gtts           # For text-to-speech

# Save dependencies
pip freeze > requirements.txt
```

Create Files

Step 1: Create main.py

```
python
```

```
# main.py
from src.motion_detector import MotionDetector
import os
from dotenv import load_dotenv

load_dotenv()

if __name__ == "__main__":
    CAMERA_ID = os.getenv('CAMERA_ID', '0')
    API_ENDPOINT = os.getenv('API_ENDPOINT', 'http://localhost:3000/api/motion')

    # Convert to int if it's a number
    try:
        CAMERA_ID = int(CAMERA_ID)
    except ValueError:
        pass # It's an RTSP URL

    detector = MotionDetector(camera_id=CAMERA_ID, api_endpoint=API_ENDPOINT)
    detector.run()
```

Step 2: Copy motion detector code

```
bash

# Copy the code from artifact "motion_detection_py" to:
# src/motion_detector.py
```

Step 3: Create .env file

```
bash

# .env
CAMERA_ID=0
# For IP camera use: CAMERA_ID=rtsp://admin:password@192.168.1.100:554/stream
API_ENDPOINT=http://localhost:3000/api/motion
MOTION_SENSITIVITY=25
NOTIFICATION_COOLDOWN=300
```

Step 4: Create audio generation script

```
python
```

```
# scripts/generate_audio.py
from gtts import gTTS
import os

def generate_welcome_message():
    text = "Welcome to our shop! Please wait, I will be there in 5 minutes."
    tts = gTTS(text=text, lang='en')

    # Create audio directory if it doesn't exist
    os.makedirs('./audio', exist_ok=True)

    # Save the audio file
    tts.save('./audio/welcome_message.mp3')
    print("Audio file generated successfully!")

if __name__ == "__main__":
    generate_welcome_message()
```

Step 5: Create Dockerfile

dockerfile


```
# Dockerfile
```

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

```
# Install system dependencies
```

```
RUN apt-get update && apt-get install -y \
```

```
libgl1-mesa-glx \
```

```
libgl1-mesa-glx \
```

```
libsm6 \
```

```
libxext6 \
```

```
libxrender-dev \
```

```
&& rm -rf /var/lib/apt/lists/*
```

```
# Copy requirements and install
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy application code
```

```
COPY . .
```

```
# Run the application
```

```
CMD ["python", "main.py"]
```

3. REACT NATIVE MOBILE APP

Create React Native (Expo) Project

```
bash
```

```
npx create-expo-app mobile-app
```

```
cd mobile-app
```

Complete Folder Structure

```
mobile-app/
```

```
├── app/
```

```
│   ├── (auth)/
```

```
│   │   ├── login.tsx
```

```
│   │   └── register.tsx
```

```
| |   └─ _layout.tsx
| |   └─ (tabs)/
| |     └─ index.tsx           # Home/Dashboard
| |     └─ cameras.tsx        # Camera list
| |     └─ events.tsx         # Motion events
| |     └─ notifications.tsx   # Notifications
| |     └─ _layout.tsx
| |   └─ camera/
| |     └─ [id].tsx           # Camera details
| |     └─ add.tsx            # Add camera
| |   └─ event/
| |     └─ [id].tsx           # Event details
| |   └─ _layout.tsx
| |   └─ +not-found.tsx
| └─ components/
|   └─ ui/
|     └─ Button.tsx
|     └─ Card.tsx
|     └─ Input.tsx
|   └─ CameraCard.tsx
|   └─ EventCard.tsx
|   └─ NotificationItem.tsx
|   └─ LiveStreamView.tsx
| └─ services/
|   └─ api.ts                 # API client
|   └─ notifications.ts       # Push notifications setup
|   └─ storage.ts             # AsyncStorage utilities
| └─ hooks/
|   └─ useAuth.ts
|   └─ useCameras.ts
|   └─ useEvents.ts
|   └─ useNotifications.ts
| └─ contexts/
|   └─ AuthContext.tsx
|   └─ NotificationContext.tsx
| └─ types/
|   └─ index.ts
| └─ constants/
|   └─ config.ts
| └─ utils/
|   └─ helpers.ts
| └─ assets/
|   └─ images/
|   └─ fonts/
```

```
|— app.json
|— package.json
|— tsconfig.json
|— README.md
```

Install Dependencies

```
bash
```

```
cd mobile-app
```

```
# Install core dependencies
```

```
npx expo install expo-notifications expo-device expo-constants
```

```
npx expo install expo-camera expo-av
```

```
npx expo install @react-navigation/native
```

```
npx expo install expo-secure-store expo-image-picker
```

```
# Install additional utilities
```

```
npm install axios zustand date-fns
```

```
npm install @tanstack/react-query
```

```
# Icons
```

```
npm install @expo/vector-icons
```

Create Files

Step 1: Create notification service

Create file: `services/notifications.ts`

```
typescript
```

```

import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';
import { Platform } from 'react-native';
import Constants from 'expo-constants';

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: true,
  }),
});

export async function registerForPushNotificationsAsync() {
  let token;

  if (Platform.OS === 'android') {
    await Notifications.setNotificationChannelAsync('motion_alerts', {
      name: 'Motion Alerts',
      importance: Notifications.AndroidImportance.MAX,
      vibrationPattern: [0, 250, 250, 250],
      lightColor: '#FF231F7C',
    });
  }

  if (Device.isDevice) {
    const { status: existingStatus } = await Notifications.getPermissionsAsync();
    let finalStatus = existingStatus;

    if (existingStatus !== 'granted') {
      const { status } = await Notifications.requestPermissionsAsync();
      finalStatus = status;
    }

    if (finalStatus !== 'granted') {
      alert('Failed to get push token for push notification!');
      return;
    }

    token = (await Notifications.getExpoPushTokenAsync({
      projectId: Constants.expoConfig?.extra?.eas?.projectId,
    })).data;
  } else {

```

```
    alert('Must use physical device for Push Notifications');  
  }  
  
  return token;  
}
```

Step 2: Create API client

Create file: `services/api.ts`

typescript

```
import axios from 'axios';
import * as SecureStore from 'expo-secure-store';

const API_URL = 'http://YOUR_BACKEND_URL/api'; // Change this

const api = axios.create({
  baseURL: API_URL,
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Add auth token to requests
api.interceptors.request.use(async (config) => {
  const token = await SecureStore.getItemAsync('authToken');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export const authAPI = {
  login: (email: string, password: string) =>
    api.post('/auth/login', { email, password }),
  register: (email: string, password: string, name: string) =>
    api.post('/auth/register', { email, password, name }),
  logout: () => api.post('/auth/logout'),
};

export const cameraAPI = {
  getAll: () => api.get('/cameras'),
  getById: (id: string) => api.get(`/cameras/${id}`),
  create: (data: any) => api.post('/cameras', data),
  update: (id: string, data: any) => api.put(`/cameras/${id}`, data),
  delete: (id: string) => api.delete(`/cameras/${id}`),
};

export const eventAPI = {
  getAll: (limit?: number) => api.get('/events', { params: { limit } }),
  getById: (id: string) => api.get(`/events/${id}`),
};
```

```
export const notificationAPI = {
  getAll: () => api.get('/notifications'),
  markAsRead: (id: string) => api.post(`/notifications/${id}/read`),
  savePushToken: (token: string) =>
    api.post('/notifications/token', { expoPushToken: token }),
};

export default api;
```

Step 3: Create main app file

Create file: `app/(tabs)/index.tsx`

typescript

```
import { View, Text, FlatList, RefreshControl, StyleSheet } from 'react-native';
import { useState, useEffect } from 'react';
import { eventAPI } from '@services/api';
import EventCard from '@components/EventCard';
import { registerForPushNotificationsAsync } from '@services/notifications';
import * as Notifications from 'expo-notifications';

export default function HomeScreen() {
  const [events, setEvents] = useState([]);
  const [refreshing, setRefreshing] = useState(false);
  const [expoPushToken, setExpoPushToken] = useState("");

  useEffect(() => {
    // Register for push notifications
    registerForPushNotificationsAsync().then(token => {
      setExpoPushToken(token || "");
      // Save token to backend
      if (token) {
        notificationAPI.savePushToken(token);
      }
    });

    // Listen for notifications
    const subscription = Notifications.addNotificationReceivedListener(notification => {
      console.log('Notification received:', notification);
      loadEvents(); // Refresh events when notification received
    });

    loadEvents();

    return () => subscription.remove();
  }, []);

  const loadEvents = async () => {
    try {
      const response = await eventAPI.getAll(20);
      setEvents(response.data.events);
    } catch (error) {
      console.error('Error loading events:', error);
    }
  };

  const onRefresh = async () => {
```



```
setRefreshing(true);
await loadEvents();
setRefreshing(false);
};

return (
  <View style={styles.container}>
    <Text style={styles.title}>Recent Motion Events</Text>
    <FlatList
      data={events}
      keyExtractor={({item}) => item.id}
      renderItem={({ item }) => <EventCard event={item} />}
      refreshControl={
        <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
      }
    />
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 16,
  },
});
```

COMPLETE SETUP COMMANDS

1. Setup Everything

```
bash
```

```
# Create root directory
mkdir cctv-motion-detection
cd cctv-motion-detection

# Setup Next.js Backend
npx create-next-app@latest backend-nextjs
cd backend-nextjs
npm install @prisma/client firebase-admin bcryptjs jsonwebtoken next-auth
npm install -D prisma typescript
npx prisma init
# Copy schema and create files as shown above
cd ..

# Setup Python Service
mkdir python-motion-service
cd python-motion-service
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install opencv-python numpy requests pygame python-dotenv gtts
# Create files as shown above
cd ..

# Setup Mobile App
npx create-expo-app mobile-app
cd mobile-app
npx expo install expo-notifications expo-device expo-constants
npm install axios zustand
# Create files as shown above
cd ..
```

2. Run All Services

```
bash
```

Terminal 1: Next.js Backend

```
cd backend-nextjs
```

```
npx prisma migrate dev --name init
```

```
npm run dev
```

Terminal 2: Python Motion Service

```
cd python-motion-service
```

```
source venv/bin/activate
```

```
python scripts/generate_audio.py
```

```
python main.py
```

Terminal 3: Mobile App

```
cd mobile-app
```

```
npx expo start
```

This complete guide provides the exact folder structure and where each piece of code should go!